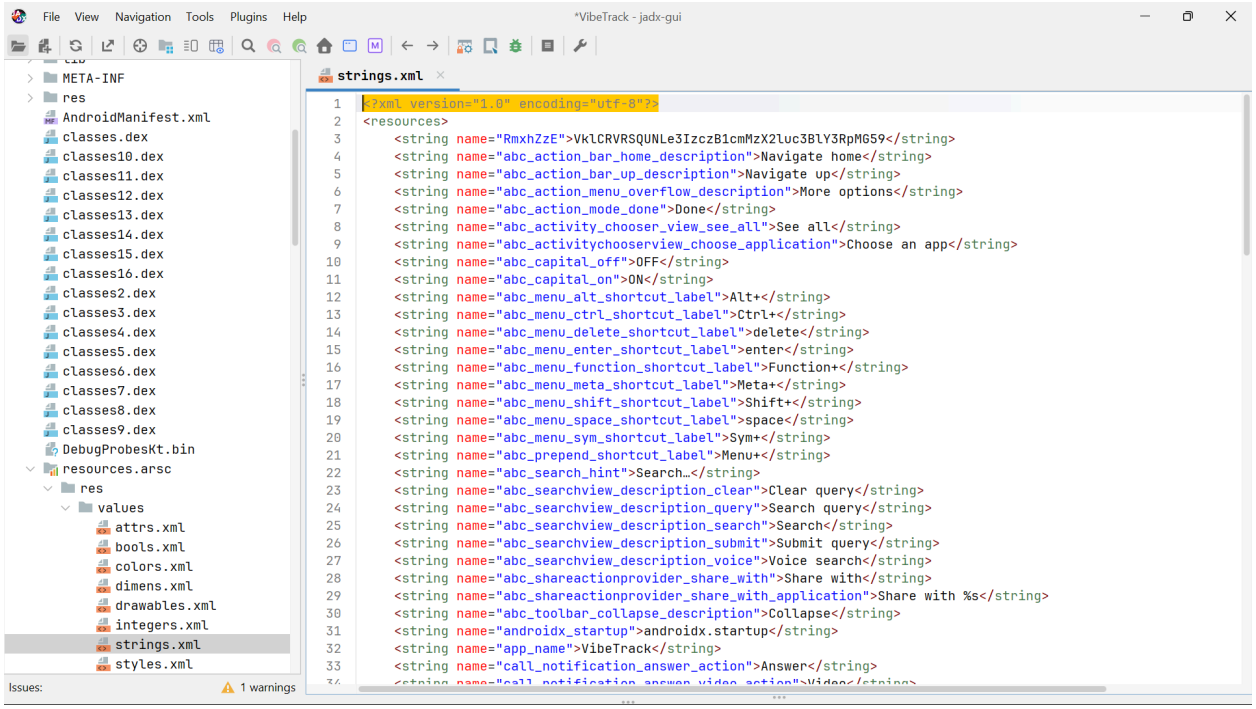
VibeTrack CTF – Writeup

Prerequisites

- APK file: VibeTrack.apk
- Tools:
  - adb (Android Debug Bridge) – for interacting with an emulator/device
  - jadx – for decompilation
  - sqlite3 – for querying the database
  - python3 with cryptography and Pillow – for decryption and steganography
  - A terminal (Linux/macOS) or WSL (Windows)
  - Base64 - encoding and decoding

Flag 1 – Resource Inspection

Flag: VIBETRACK{r3s0urc3_inspecti0n}



Steps

1. Decompile the APK
2. Navigate to VibeTrack/res/values/strings.xml.
3. Find the row with encoded strings, decode both the key and the value
   Output: VIBETRACK{r3s0urc3_inspecti0n}

Flag 2 – Database Extraction

Flag: VIBETRACK{db_extract}

Steps

1. adb exec-out run-as com.vibetrack.habits cat databases/vibetrack_database > vibetrack_database.db
2. Open the database with sqlite3:
3. sqlite3 vibetrack_database.db
4. Query the hidden habit:
5. SELECT description FROM habits WHERE name = 'FLAG';
   Result: VklCRVRSQUNLe2RiX2V4dHJhY3R9
6. Decode Base64:
7. bash
8. echo "VklCRVRSQUNLe2RiX2V4dHJhY3R9" | base64 -d
   Output: VIBETRACK{db_extract}

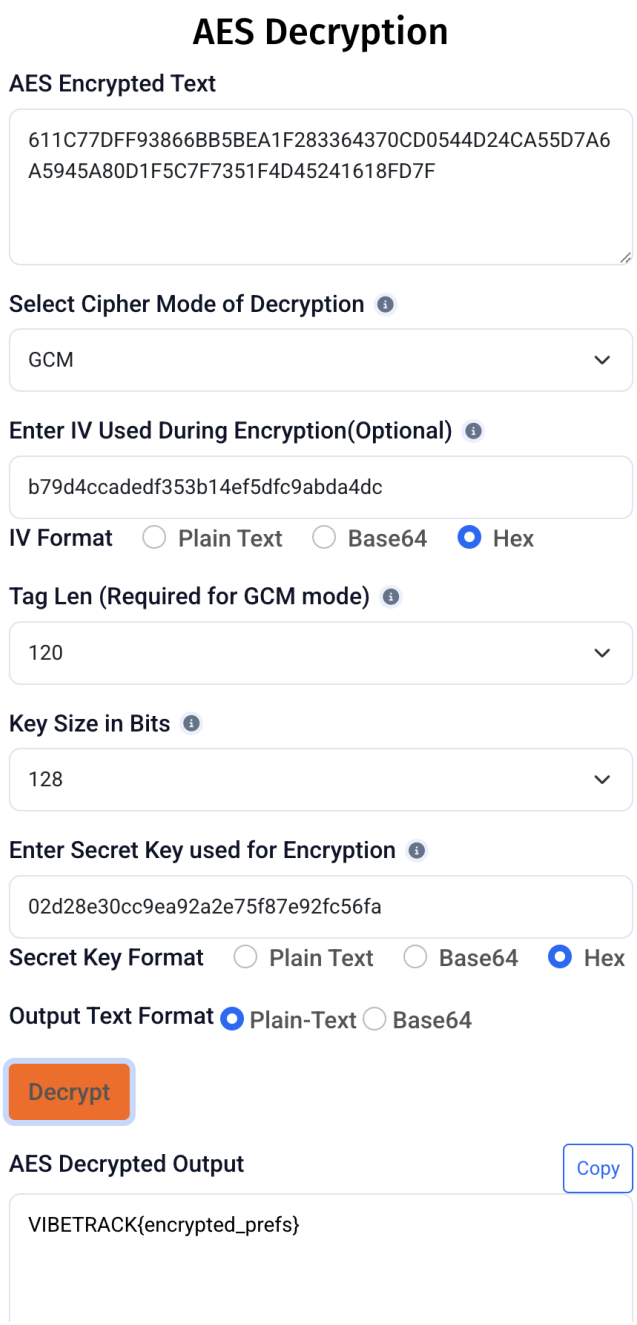
```
emu64a16k:/data/user/0/com.vibetrack.habits/databases $ sqlite3 vibetrack_database "SELECT description FROM habits WHERE name='FLAG';"
VklCRVRSQUNLe2RiX2V4dHJhY3R9
emu64a16k:/data/user/0/com.vibetrack.habits/databases $
```

Flag 3 – Intermediate: Encrypted Database with Key in Manifest

Flag: VIBETRACK{encrypted_db}

Steps

1. open AndroidManifest.xml.
2. Find the <meta-data> tag:
3. <meta-data android:name="com.vibetrack.habits.ENCRYPTION_KEY" android:value="VmlCZVRyYWNrU2VjcmV0MjAyNA==" />
4. Decode the key:
5. echo "VmlCZVRyYWNrU2VjcmV0MjAyNA==" | base64 -d
   Output: VibeTrackSecret2024
6. Extract the database as in Flag 2, then dump the flags table:
7. bash
8. sqlite3 vibetrack_database.db "SELECT hex(encryptedFlag) FROM flags WHERE id=1;" > flag3.hex
9. Convert hex to binary (e.g., using xxd -r -p flag3.hex > flag3.bin).
10. Decrypt using Python (AES-GCM, key derived with SHA-256):
11. Output: VIBETRACK{encrypted_db}



Flag 4 – Intermediate: Native Code XOR

Flag: VIBETRACK{xor_native}

Steps

1. locate lib/arm64-v8a/libflag4.so (or lib/x86/libflag4.so for emulators).
2. Use strings or a hex editor to find the encrypted array and the key.

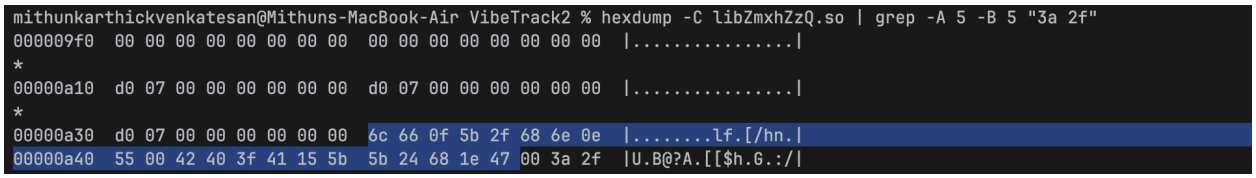
- Encrypted bytes (example from the library):
- 52 5A 5C 4A 5D 54 51 5B 54 5A 53 52 58 4F 56 5F 5C 5D 53 5A 54 51 5E 5D 4E 5C 5D 55 5E 58 4E 5F 5C 4F 56 5D 5C 4F 5D
- Key: 3A 2F 4D 1E 7B

3. XOR decrypt with Python:
4. python

```
encrypted = bytes([
    0x6c, 0x66, 0x0f, 0x5b, 0x2f, 0x68, 0x6e, 0x0e, 0x55, 0x00, 0x42, 0x40, 0x3f, 0x41, 0x15, 0x5b, 0x5b, 0x24, 0x68, 0x1e, 0x47
])
key = bytes([0x3A, 0x2F, 0x4D, 0x1E, 0x7B])
flag = ''.join(chr(encrypted[i] ^ key[i % len(key)]) for i in range(len(encrypted)))
```

5. print(flag)

   Output: VIBETRACK{xor_native}



```
mithunkarthickvenkatesan@Mithuns-MacBook-Air VibeTrack2 % hexdump -C libZmxhZzQ.so | grep -A 5 -B 5 "3a 2f"
000009f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000a10  d0 07 00 00 00 00 00 00  d0 07 00 00 00 00 00 00  |................|
*
00000a30  d0 07 00 00 00 00 00 00  6c 66 0f 5b 2f 68 6e 0e  |........lf.[/hn.|
00000a40  55 00 42 40 3f 41 15 5b  5b 24 68 1e 47 00 3a 2f  |U.B@?A.[[$h.G.:/|
```

```
Password:
(venv) %n@%m %1~ %# python3 k.py
Flag: VIBETRACK{xor_native}
(venv) %n@%m %1~ %#
```

🏁 Flag 5 – Advanced: Steganography in Image

Flag: VIBETRACK{stegan0graphy}

Steps

1. locate res/raw/hidden_flag.png.
2. Use Python to extract the LSB from the blue channel of the first 200×200 pixels:
3. python

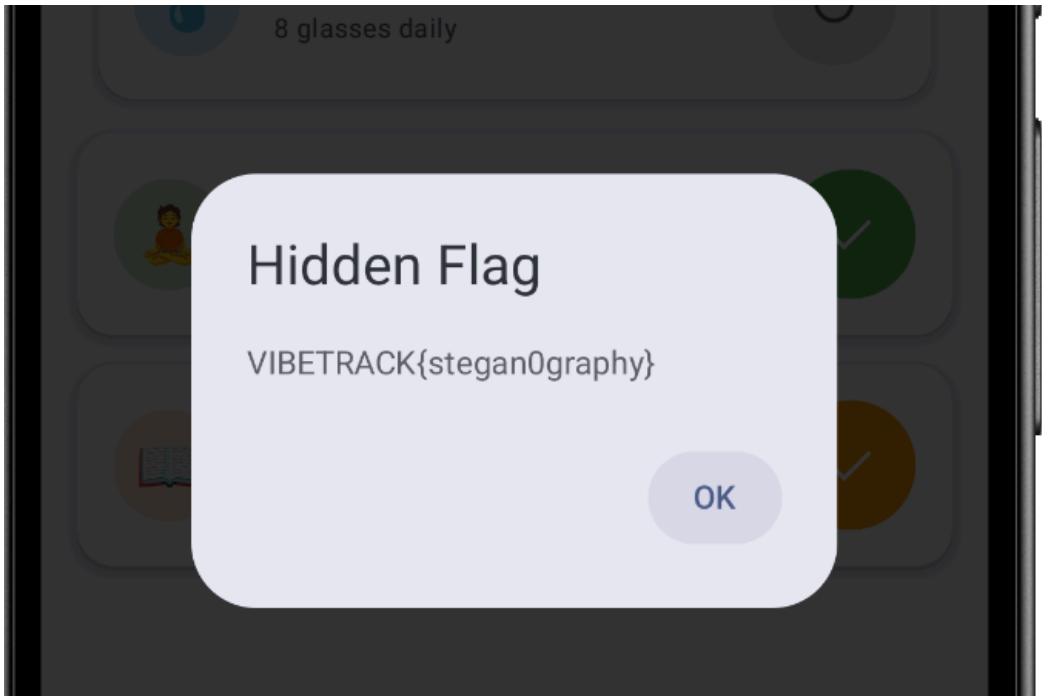
```
from PIL import Image

img = Image.open("hidden_flag.png")
pixels = img.load()
bits = []
for y in range(200):
    for x in range(200):
        r, g, b = pixels[x, y]
```

```
        bits.append(b & 1)
        if len(bits) >= 500:
            break
    if len(bits) >= 500:
        break

# Convert bits to bytes (MSB first)
flag_bytes = []
for i in range(0, len(bits), 8):
    if i+8 > len(bits):
        break
    byte = 0
    for j in range(8):
        byte = (byte << 1) | bits[i+j]
    if byte == 0:
        break
    flag_bytes.append(byte)
```

4. print(bytes(flag_bytes).decode())
   Output: VIBETRACK{stegan0graphy}
   Alternative: Long-press the "Today's Habits" title in the running app – a dialog will show the flag.



🏁 Flag 6 – Advanced: Hidden BroadcastReceiver

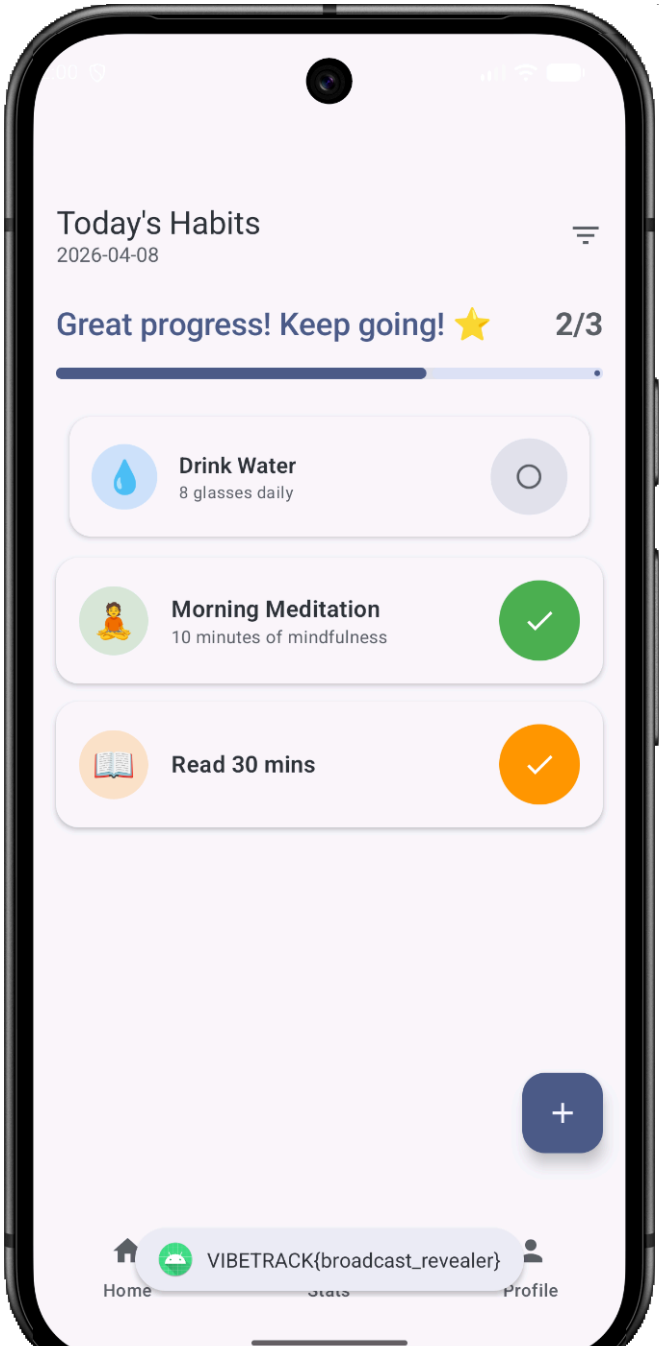
Flag: VIBETRACK{broadcast_revealer}

Steps

1. inspect AndroidManifest.xml. Find:
2.

```
<receiver android:name=".FlagReceiver" android:exported="true">
  <intent-filter>
    <action android:name="com.vibetrack.habits.REVEAL_FLAG" />
  </intent-filter>
```

3. </receiver>
4. Send the broadcast:
5.
6.
```
adb shell am broadcast -a com.vibetrack.habits.REVEAL_FLAG -n com.vibetrack.habits/.FlagReceiver
```
7.
8. A toast will appear with the flag.
   Alternatively, decode the Base64 string from the decompiled FlagReceiver code:
9. bash
10. echo "VklCRVRSQUNLe2Jyb2FkY2FzdF9yZXZlYWxlcn0=" | base64 -d
    Output: VIBETRACK{broadcast_revealer}


Today's Habits
2026-04-08
Great progress! Keep going! ⭐
2/3
Drink Water
8 glasses daily
Morning Meditation
10 minutes of mindfulness
Read 30 mins
VIBETRACK{broadcast_revealer}
Home
Profile

🏁 Flag 7 – Advanced: Background Worker (File System)

Flag: VIBETRACK{time_based}

Steps

1. Keep the app in the foreground for at least 5 minutes (or background it – the worker will still run).
2. After 5 minutes, pull the flag file:
3. adb exec-out run-as com.vibetrack.habits cat files/flag7.txt
   Output: VIBETRACK{time_based}
   If the file does not exist, wait longer or restart the app and wait again.

```
sh-3.2# adb exec-out run-as com.vibetrack.habits cat files/flag7.txt
VIBETRACK{time_based}sh-3.2#
```

🏁 Flag 8 – Advanced: Deep Link Activity

Flag: VIBETRACK{reflection_deeplink}

Steps

1. inspect AndroidManifest.xml for the DeepLinkActivity:
2.

```
<activity android:name=".DeepLinkActivity" android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="vibetrack" android:host="flag" android:pathPrefix="/reveal" />
  </intent-filter>
```

3. </activity>
4. Trigger the deep link using ADB:
5. adb shell am start -a android.intent.action.VIEW -d "vibetrack://flag/reveal" com.vibetrack.habits
6. The activity opens and displays the flag.

```
sh-3.2# adb shell am start -a android.intent.action.VIEW -d "vibetrack://flag/re
veal" com.vibetrack.habits
Starting: Intent { act=android.intent.action.VIEW dat=vibetrack://flag/... pkg=c
om.vibetrack.habits }
```

VIBETRACK{reflection_deeplink}

Summary Table

Flag	Technique	Tools Used
1	Base64 string in resources	apktool, base64
2	Hidden habit in Room database	adb, sqlite3
3	AES-GCM encryption, key in manifest	apktool, python (cryptography)
4	XOR in native library	hexdump, python
5	LSB steganography in PNG	apktool, python (Pillow)
6	Broadcast receiver	adb, base64

7	WorkManager file write	adb, wait 5 minutes
8	Deep link activity	adb, manifest analysis